Causal Inference with Neural Networks: TARNet, CFR, and Dragonnet

Author: Peter Zhang
Date: May 2026

Estimating the Individual Treatment Effect (ITE) from observational data is a fundamental challenge in causal inference. Unlike standard machine learning, which focuses on predicting outcomes based on passive correlation, causal inference aims to answer counterfactual questions: *What would have happened if we had intervened differently?* For example, in medicine, we want to know whether a patient would recover faster if given a specific treatment compared to a control.

The major hurdle is that we never observe both outcomes for the same individual. This is known as the **fundamental problem of causal inference**. Because the counterfactual is always unobserved, we must rely on advanced models to estimate the missing potential outcomes.

Notebook Catalog

---

1. Problem Setup and Motivation

Recall that in the framework of the Rubin Causal Model, we characterize each individual by a set of background covariates X, a binary treatment assignment $T \in \{0, 1\}$, and an observed outcome Y. For every individual, there exist two potential outcomes: $Y(0)$ represents the potential outcome under the control condition, and $Y(1)$ represents the potential outcome under the active treatment condition.

Crucially, the observed outcome is a combination of these potential outcomes, dynamically selected by the actual treatment assignment:

$$Y = (1 - T)Y(0) + TY(1)$$

Our goal is to estimate the **Individual Treatment Effect (ITE)**, denoted as $\tau(X) = \mathbb{E}[Y(1) - Y(0) \mid X]$, which measures the expected change in the potential

outcome for an individual with covariates X. Aggregating this across the population yields the **Average Treatment Effect (ATE)**, defined as $\text{ATE} = \mathbb{E}[Y(1) - Y(0)]$.

The Limits of Classical Meta-Learners

Traditional machine learning approaches generally tackle this problem using two standard meta-learners, both of which suffer from distinct structural limitations:

The **S-Learner (Single Learner)** concatenates the covariates and the treatment assignment into a single feature vector $[X, T]$ and trains a single model to predict the outcome Y. While conceptually simple, this approach often fails in high-dimensional settings. Because the binary treatment indicator T is only a single feature among many dimensions in X, deep neural networks frequently regularize away or ignore the treatment variable altogether. This leads to treatment effect estimates that collapse to zero, as the model fails to capture the unique influence of the intervention.

Conversely, the **T-Learner (Two Learners)** trains two entirely separate models: one model $\mu_0(X)$ to predict the control outcome and another model $\mu_1(X)$ to predict the treated outcome. While this prevents the treatment variable from being ignored, it is highly data-inefficient. Because the two models share no parameters, they cannot learn from shared underlying relationships between the covariates X and the outcome Y that remain invariant across both treatment groups. This is particularly problematic in observational datasets where one treatment group may have very sparse coverage in certain regions of the covariate space.

Shared Representation Learning as the Bridge

To overcome these structural limitations, modern deep learning architectures learn a **shared latent representation** $\Phi(X)$ that is common to both treatment groups, and then branch into separate, treatment-specific **hypothesis heads** (or outcome heads) to predict the outcomes $\hat{Y}(0)$ and $\hat{Y}(1)$. This shared representation structure forces the network to learn a rich, shared feature space from the entire dataset (solving the T-learner's data inefficiency), while the physical separation of the prediction heads guarantees that the treatment assignment dynamically drives distinct outcomes (solving the S-learner's treatment-ignorance problem).

In this notebook, we explore the progression of this paradigm. We begin by evaluating **TARNet** (Treatment-Agnostic Representation Network), which establishes this shared representation framework. We then implement **Counterfactual Regression (CFR)**, which adds balance regularization (MMD or Wasserstein distance) to the representation space to address selection bias. Finally, we implement **Dragonnet**, which uses a propensity score head as a targeted regularizer to preserve vital confounding variables.

Reproducibility Setup

To ensure that all experiments, representation visualizations (PCA/t-SNE), neural network weight initializations are reproducible and deterministic, we configure the global random

seeds and environment settings below. This should guarantee consistent in-sample and out-of-sample PEHE results across different runs.

In [1]:
```python
import os
import random
import numpy as np
import torch

def set_deterministic(seed=42):
    # Set Python built-in random seed
    random.seed(seed)

    # Set Python hash seed
    os.environ['PYTHONHASHSEED'] = str(seed)

    # Set NumPy random seed
    np.random.seed(seed)

    # Set PyTorch random seed
    torch.manual_seed(seed)

    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

    print(f"Global random seed set to {seed}. Deterministic backends configured

# Apply deterministic settings
set_deterministic(42)
```

```
Global random seed set to 42. Deterministic backends configured successfully!
```

2. Dataset

To see these neural network architectures in action, we are going to use the **Infant Health and Development Program (IHDP)** dataset. Originally a randomized clinical trial focused on the effects of home visits on low-birth-weight infants, the IHDP dataset has become a gold-standard benchmark in the causal inference community.

Why is it so widely used? Because it is *semi-simulated*. The background covariates X and treatment assignments T are from the real clinical trial, but the potential outcomes $Y(0)$ and $Y(1)$ are simulated using known mathematical functions. This simulation gives us something we can never get in the real world: **access to the true ground-truth counterfactuals!**

Because we have the true counterfactuals, we can compute the **PEHE (Precision in Estimation of Heterogeneous Effects)** to measure exactly how well our models predict the Individual Treatment Effect (ITE):

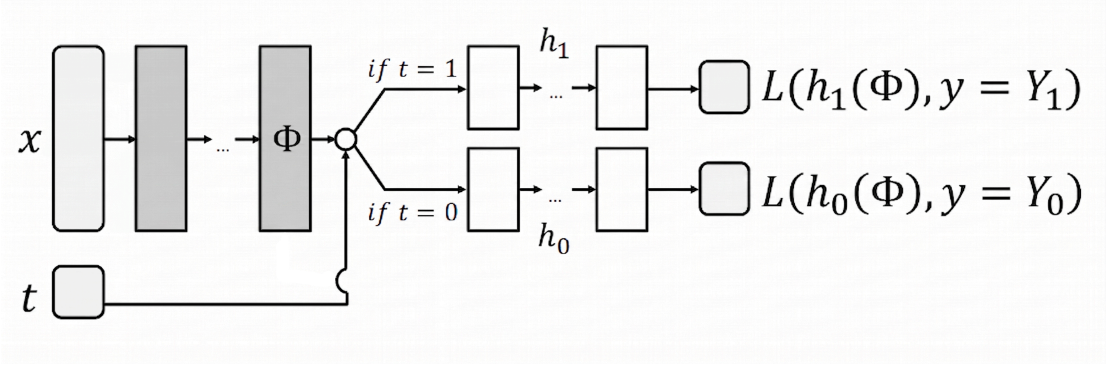
$$\text{PEHE} = \frac{1}{N}\sum_{i=1}^{N}\left(\tau(X_i) - \hat{\tau}(X_i)\right)^2$$

> [!WARNING] **A Quick Disclaimer Before We Begin**: The official IHDP benchmark actually consists of **1000 distinct simulated outcome surfaces** (realizations) to make sure evaluations are statistically robust. Because training on all 1000 surfaces would take some time, **we will focus on a single realization (`rep = 0`)** for this walkthrough. This keeps our computations fast and lets us visualize our results beautifully, but keep in mind that full research studies average their results across all 1000 replications!

3. Treatment-Agnostic Representation Network (TARNet)

Intuition & Formulation

Let's kick things off with our baseline model: **TARNet**! Developed by Shalit et al., TARNet eleganty solves the S-learner's "treatment-ignorance" problem by physically separating the prediction heads while still letting them learn from a shared representation of the background features.



Here is how the magic works:

1. First, we feed the covariates X into a shared neural network $\Phi(X)$ to extract general, dense representations.
2. Next, the network branches into two independent "hypothesis heads": $h_0(\Phi(X))$ to predict the control outcome $\hat{Y}(0)$ and $h_1(\Phi(X))$ to predict the treated outcome $\hat{Y}(1)$.

During training, we minimize the factual Mean Squared Error (MSE) on the observed (factual) outcomes:

$$\mathcal{L}_{TARNet} = \frac{1}{N}\sum_{i=1}^{N}\left(Y_i - h_{T_i}(\Phi(X_i))\right)^2$$

Why It Excels

TARNet is incredibly **data-efficient**! By sharing the representation layers Φ, it learns general, high-quality predictive patterns from the entire dataset, regardless of treatment.

At the same time, the split outcome heads ensure the model never "forgets" the treatment variable. It's a balanced starting point for causal representation learning.

Step 1: The Shared Representation

Let's build the foundation of TARNet: the **shared latent representation**. Instead of predicting the outcome directly from the raw covariates X, we first pass the input through a set of shared layers to extract a dense, highly expressive feature embedding:

$$\Phi(X) = \text{SharedLayers}(X)$$

This ensures that the network extracts general features that are useful for understanding the patient's background, regardless of which treatment they received. In PyTorch, we can implement this using a few simple dense layers!

In [2]:
```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SharedRepresentation(nn.Module):
    r"""
    Learns a shared latent representation \Phi(X) from the input covariates.

    This module maps the high-dimensional background covariates to a lower-dimen
    dense representation. The learned representation is shared between the treat
    and control hypothesis heads, allowing the model to learn invariant features
    that generalize across both groups.
    """
    def __init__(self, input_dim=25, hidden_dim=200):
        super().__init__()

        # Dense network layers with ELU activations to learn non-linear shared f
        self.network = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ELU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ELU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ELU()
        )

    def forward(self, x):
        h = self.network(x)

        # CFRNet applies L2 normalization to keep the latent space bounded,
        # which is crucial for stable IPM (MMD / Wasserstein) penalty calculatio
        return F.normalize(h, p=2, dim=1)
```

Step 2: The Treatment Split

Now that we have our shared representation $\Phi(X)$, it's time to branch out! We'll build two separate **Hypothesis Heads** (outcome branches) to estimate potential outcomes:

$$\hat{Y}(0) = h_0(\Phi(X))$$

$$\hat{Y}(1) = h_1(\Phi(X))$$

During the forward pass, we will route our data based on the observed treatment assignment T. The untreated (counterfactual) branch will then be ignored during backpropagation. This dynamic routing ensures the model never ignores the active intervention, keeping our estimations sharp and accurate.

In [3]:
```python
class HypothesisHead(nn.Module):
    r"""
    Predicts the potential outcome Y(t) given the shared latent representation \

    In the causal graph, separate hypothesis heads are trained for the control (
    and treated (t=1) branches to prevent the model from ignoring the treatment
    """
    def __init__(self, input_dim=200, hidden_dim=100):
        super().__init__()

        # Multi-layer regression network translating shared representation to sc
        self.network = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ELU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ELU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ELU(),
            nn.Linear(hidden_dim, 1)
        )

    def forward(self, phi):
        return self.network(phi)

class TARNet(nn.Module):
    r"""
    Treatment-Agnostic Representation Network (TARNet).

    TARNet splits the prediction problem by learning a shared feature extraction
    and then routing the representation to two independent hypothesis heads.
    """
    def __init__(self):
        super().__init__()
        self.phi = SharedRepresentation(input_dim=25, hidden_dim=200)
        self.h0 = HypothesisHead(input_dim=200, hidden_dim=100)
        self.h1 = HypothesisHead(input_dim=200, hidden_dim=100)

    def forward(self, x, t):
        phi = self.phi(x)
        y0_hat = self.h0(phi)
        y1_hat = self.h1(phi)

        # Dynamically route the factual prediction based on the observed treatme
        y_hat = (1 - t) * y0_hat + t * y1_hat
        return y_hat, y0_hat, y1_hat
```

Step 3: The Objective Function

To train TARNet, we use the **Factual Mean Squared Error (MSE)**. Since we only observe the outcome of the treatment that was actually assigned, we calculate our loss exclusively

on those observed factual pairs:

$$\mathcal{L}_{\text{TARNet}} = \frac{1}{N}\sum_{i=1}^{N}\Big(y_i - \hat{Y}(T_i)\Big)^2$$

Where $\hat{Y}(T_i) = h_{T_i}(\Phi(x_i))$.

In [4]:
```
def tarnet_loss(y_true, y_hat):
    r"""
    Computes the standard Mean Squared Error (MSE) loss on the observed outcomes

    This objective guides the model to fit the factual outcomes without regulari
    the representation space.
    """
    criterion = nn.MSELoss()
    return criterion(y_hat, y_true)
```

Step 4: Data Fetching and Preprocessing

Now we'll be loading our IHDP data! We are going to use the official full 1000-realization dataset exactly as configured in the original CFRNet paper.

Let's load the training and test splits, check for GPU acceleration, and get our data dimensions ready.

In [5]:
```
import numpy as np
import torch
from torch.utils.data import TensorDataset, DataLoader

# Select GPU if available, otherwise fall back to CPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Load the full IHDP causal benchmark dataset
d_train = np.load('ihdp_full/ihdp_npci_1-1000.train.npz')
d_test = np.load('ihdp_full/ihdp_npci_1-1000.test.npz')

# Extract and prepare training arrays
x_train_all = torch.FloatTensor(d_train['x']).to(device)                # Shape (
t_train_all = torch.FloatTensor(d_train['t']).unsqueeze(2).to(device)  # Shape (
yf_train_all = torch.FloatTensor(d_train['yf']).unsqueeze(2).to(device)# Shape (
mu0_train_all = d_train['mu0']                                          # Shape (
mu1_train_all = d_train['mu1']                                          # Shape (

# Extract and prepare test arrays
x_test_all = torch.FloatTensor(d_test['x']).to(device)                  # Shape (
t_test_all = torch.FloatTensor(d_test['t']).unsqueeze(2).to(device)
yf_test_all = torch.FloatTensor(d_test['yf']).unsqueeze(2).to(device)
mu0_test_all = d_test['mu0']
mu1_test_all = d_test['mu1']

num_replications = x_train_all.shape[2]
print(f"Loaded {num_replications} replications. Train shape: {x_train_all.shape}
```

```
Using device: cuda
Loaded 1000 replications. Train shape: torch.Size([672, 25, 1000]), Test shape: t
orch.Size([75, 25, 1000])
```

Step 5: Training Loop

Let's bring TARNet to life! For this demonstration, we'll select the first realization (`rep = 0`) to train our model. We will implement a standard PyTorch training loop, running for exactly 3000 iterations using the Adam optimizer.

In [6]:
```python
# Hyperparameters and Realization configuration
iterations = 200
batch_size = 100
rep = 0 # Index of the replication realization to train

print(f"Training on Realization {rep}...")

# Slice replication data
x_tr, t_tr, y_tr = x_train_all[:, :, rep], t_train_all[:, rep, :], yf_train_all[
x_te, t_te = x_test_all[:, :, rep], t_test_all[:, rep, :]

# PyTorch DataLoader for mini-batch training
dataset_train = TensorDataset(x_tr, t_tr, y_tr)
dataloader_train = DataLoader(dataset_train, batch_size=batch_size, shuffle=True

# Initialize TARNet model and Adam optimizer
tarnet = TARNet().to(device)
optimizer = torch.optim.Adam(tarnet.parameters(), lr=1e-3, weight_decay=1e-4)

# Core training loop over designated iterations
tarnet.train()
current_iter = 0
while current_iter < iterations:
    for batch_x, batch_t, batch_y in dataloader_train:
        if current_iter >= iterations:
            break

        optimizer.zero_grad()
        y_hat, _, _ = tarnet(batch_x, batch_t)
        loss = tarnet_loss(batch_y, y_hat)
        loss.backward()
        optimizer.step()

        if current_iter % 10 == 0:
            print("Train loss:", loss.item())

        current_iter += 1

print("Training complete!")
```

```
Training on Realization 0...
Train loss: 13.066473960876465
Train loss: 9.974353790283203
Train loss: 2.4106645584106445
Train loss: 1.191974401473999
Train loss: 1.0945591926574707
Train loss: 1.058699607849121
Train loss: 0.9869317412376404
Train loss: 1.1209983825683594
Train loss: 0.9825562834739685
Train loss: 0.8914496302604675
Train loss: 0.9418628215789795
Train loss: 0.7624578475952148
Train loss: 0.8538418412208557
Train loss: 1.0531792640686035
Train loss: 1.0400991439819336
Train loss: 0.7761399745941162
Train loss: 1.094110131263733
Train loss: 1.022463083267212
Train loss: 1.0154584646224976
Train loss: 0.9029756784439087
Training complete!
```

Step 6: Evaluation and Visualization

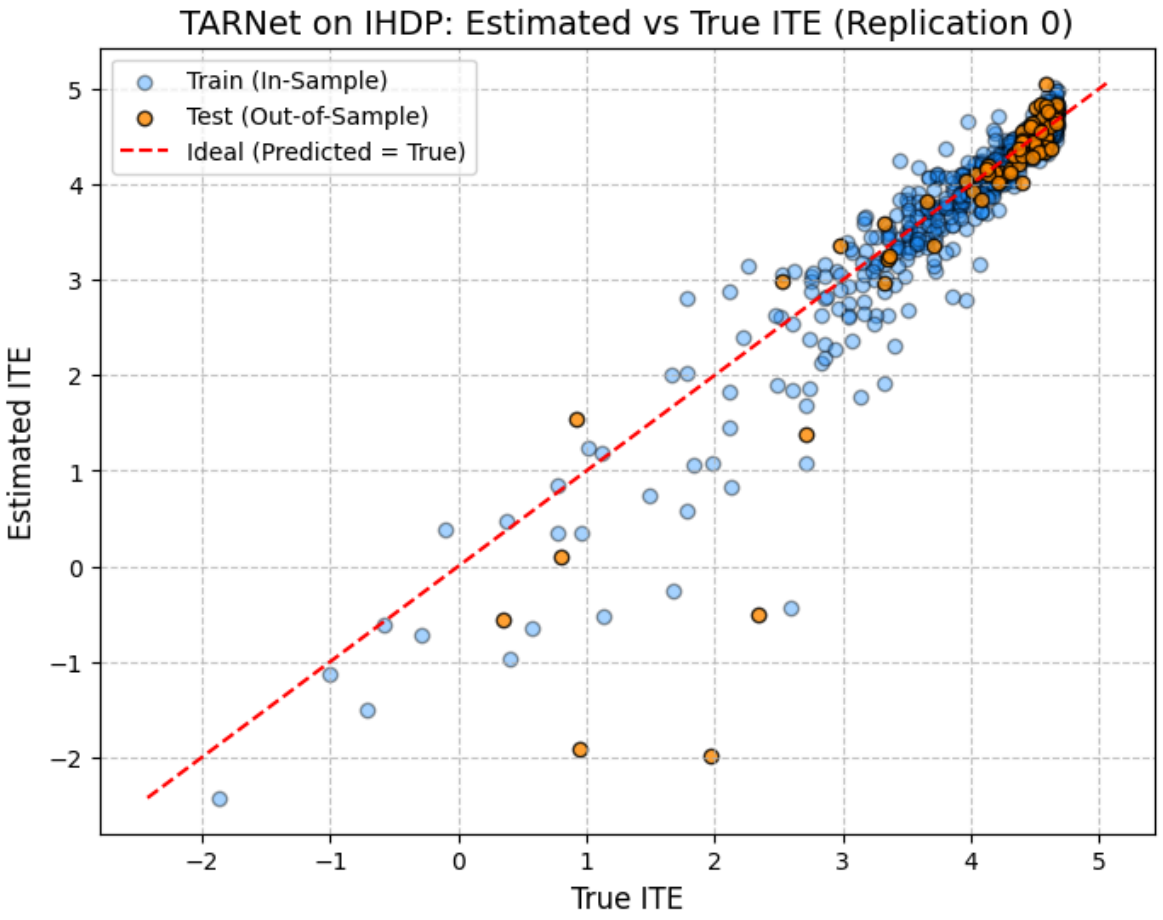
With our TARNet model successfully trained, let's see how well it estimates the **PEHE** (Mean Squared Error of the ITE) both in-sample and out-of-sample.

We'll also generate a beautiful scatter plot comparing our Estimated ITE against the True ITE. In an ideal world, all of our points would lie perfectly along the diagonal $y = x$ line ——let's see how close we get!

In [7]:
```python
import matplotlib.pyplot as plt
import numpy as np

tarnet.eval()
with torch.no_grad():
    # Evaluate In-Sample PEHE
    _, y0_hat_in, y1_hat_in = tarnet(x_tr, t_tr)
    ite_hat_in = (y1_hat_in - y0_hat_in).cpu().numpy()
    ite_true_in = mu1_train_all[:, rep:rep+1] - mu0_train_all[:, rep:rep+1]
    pehe_in = np.mean((ite_true_in - ite_hat_in)**2)

    # Evaluate Out-of-Sample PEHE
    _, y0_hat_out, y1_hat_out = tarnet(x_te, t_te)
    ite_hat_out = (y1_hat_out - y0_hat_out).cpu().numpy()
    ite_true_out = mu1_test_all[:, rep:rep+1] - mu0_test_all[:, rep:rep+1]
    pehe_out = np.mean((ite_true_out - ite_hat_out)**2)

print(f"In-Sample PEHE (MSE):     {pehe_in:.4f}")
print(f"Out-of-Sample PEHE (MSE): {pehe_out:.4f}")
print(f"Out-of-Sample RMSE:       {np.sqrt(pehe_out):.4f}")

# Plot Predicted vs True ITE scatters to visualize estimation precision
plt.figure(figsize=(8, 6))
plt.scatter(ite_true_in, ite_hat_in, alpha=0.4, color='dodgerblue', edgecolor='k
plt.scatter(ite_true_out, ite_hat_out, alpha=0.8, color='darkorange', edgecolor=
```

```
# Diagonal reference line representing ideal estimation (Predicted = True)
min_val = min(np.min(ite_true_in), np.min(ite_hat_in), np.min(ite_true_out), np.
max_val = max(np.max(ite_true_in), np.max(ite_hat_in), np.max(ite_true_out), np.
plt.plot([min_val, max_val], [min_val, max_val], 'r--', label='Ideal (Predicted

plt.title(f'TARNet on IHDP: Estimated vs True ITE (Replication {rep})', fontsize
plt.xlabel('True ITE', fontsize=12)
plt.ylabel('Estimated ITE', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

```
In-Sample PEHE (MSE):     0.0992
Out-of-Sample PEHE (MSE): 0.4994
Out-of-Sample RMSE:       0.7067
```



> [!WARNING] Disclaimer: The IHDP True ITE "Wall"
>
> When plotting Estimated vs. True ITE on the standard IHDP benchmark, you will observe that the True ITE abruptly caps out at approximately 4.5.
>
> This is a feature of the dataset, not a bug in our model. Almost all modern causal ML benchmarks use the "Response Surface B" data generating process (DGP) for IHDP. Because this DGP subtracts an exponentially growing control surface from a linearly growing treated surface, the True Treatment Effect is mathematically bounded from above. Furthermore, the dataset calibrates its offset constant specifically to force the Average Treatment Effect on the Treated (ATT) to exactly 4, locking this absolute maximum ITE ceiling into place.

> Relevant References & Code:
>
> - Original DGP Paper: Hill, J. L. (2011). Bayesian nonparametric modeling for causal inference. Journal of Computational and Graphical Statistics, 20(1), 217-240.
> - Standard Benchmark Implementation: The widespread use of this specific DGP with the ATT=4 calibration was popularized by the CFRNet paper (Shalit et al., 2017). You can see the original data generation and loading logic in their official repository: clinicalml/cfrnet on GitHub.

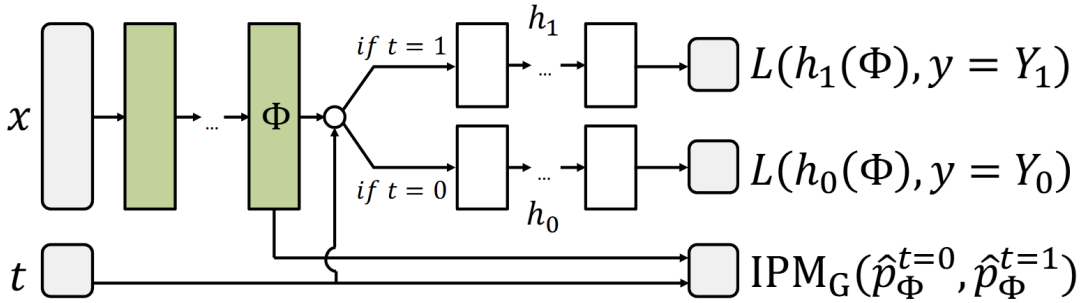
4. Counterfactual Regression (CFR)

Intuition & Formulation

TARNet is a great baseline, but it has a hidden weakness. In observational studies, people are rarely assigned treatments at random. For example, healthier patients might be less likely to receive a high-risk drug. This is known as **selection bias** (or covariate shift), and it can cause the treated and control groups to have completely different covariate distributions.

Without any regularization, TARNet might represent treated and control groups in entirely separate regions of the representation space $\Phi(X)$. When we try to predict a counterfactual, the model has to extrapolate to unseen regions, which can make predictions highly unreliable!

CFR solves this beautifully by adding an **Integral Probability Metric (IPM)** regularization penalty to the TARNet loss:



Our CFR objective function is:

$$\mathcal{L}_{CFR} = \mathcal{L}_{TARNet} + \alpha \cdot \text{IPM}\left(\{\Phi(X_i)\}_{T_i=0}, \{\Phi(X_i)\}_{T_i=1}\right)$$

We will implement two specific types of balance penalties:

1. **Maximum Mean Discrepancy (MMD)**: Measures and penalizes differences in the moments (like mean and variance) of the latent representation distributions.

2. **Wasserstein Distance**: Computes the optimal transport cost to shift one representation distribution into the other.

Why It Excels

CFR is outstanding at handling **selection bias**! It directly penalizes the divergence between the treated and control representations, and forces the network to learn a balanced latent space where the groups heavily overlap. This ensures that predicting potential outcomes is always an interpolation task, leading to much more reliable counterfactual estimates.

In [8]:
```python
# PyTorch implementations of MMD and Sinkhorn-Wasserstein balance penalties.
# Adapted directly from the original CFRNet TensorFlow codebase.

def mmd2_lin(X, t, p):
    r"""
    Computes the Linear Maximum Mean Discrepancy (MMD) balance penalty.

    This function measures the difference in the expected mean representations
    between the treated and control distributions in the shared representation s
    """
    it = (t > 0).nonzero(as_tuple=True)[0]
    ic = (t < 1).nonzero(as_tuple=True)[0]

    Xc = X[ic]
    Xt = X[it]

    # Handle edge case if a batch has no treated or no control samples
    if Xc.shape[0] == 0 or Xt.shape[0] == 0:
        return torch.tensor(0.0, device=X.device, requires_grad=True)

    mean_control = torch.mean(Xc, dim=0)
    mean_treated = torch.mean(Xt, dim=0)

    # Calculate MMD balance penalty, scaling by empirical treatment propensities
    mmd = torch.sum(torch.square(2.0 * p * mean_treated - 2.0 * (1.0 - p) * mean
    return mmd

def pdist2sq(X, Y):
    r"""
    Computes the pairwise squared Euclidean distance matrix between two sets of
    """
    nx = torch.sum(torch.square(X), dim=1, keepdim=True)
    ny = torch.sum(torch.square(Y), dim=1, keepdim=True)
    D = -2 * torch.matmul(X, Y.t()) + ny.t() + nx
    return D

def wasserstein(X, t, p, lam=10.0, its=10, sq=False, backpropT=False):
    r"""
    Computes the Sinkhorn-Knopp approximation of the Wasserstein distance.

    This measures the optimal transport cost required to align the treated repre
    distribution with the control distribution.
    """
    it = (t > 0).nonzero(as_tuple=True)[0]
    ic = (t < 1).nonzero(as_tuple=True)[0]
    Xc = X[ic]
```

```
    Xt = X[it]
    nc = float(Xc.shape[0])
    nt = float(Xt.shape[0])

    if nc == 0 or nt == 0:
        return torch.tensor(0.0, device=X.device, requires_grad=True)

    if sq:
        M = pdist2sq(Xt, Xc)
    else:
        M = torch.sqrt(torch.clamp(pdist2sq(Xt, Xc), min=1e-10))

    M_mean = torch.mean(M)
    delta = torch.max(M).detach()
    eff_lam = (lam / M_mean).detach()

    # Pad cost matrix to handle uneven support size
    row = delta * torch.ones(1, M.shape[1], device=X.device)
    Mt = torch.cat((M, row), dim=0)
    col = torch.cat((delta * torch.ones(M.shape[0], 1, device=X.device), torch.z
    Mt = torch.cat((Mt, col), dim=1)

    # Prepare target marginal distributions
    a = torch.cat((p * torch.ones(Xt.shape[0], 1, device=X.device) / nt, (1 - p)
    b = torch.cat(((1 - p) * torch.ones(Xc.shape[0], 1, device=X.device) / nc, p

    # Sinkhorn-Knopp iterative projection to solve entropic optimal transport
    Mlam = eff_lam * Mt
    K = torch.exp(-Mlam) + 1e-6
    ainvK = K / a

    u = a
    for _ in range(its):
        u = 1.0 / torch.matmul(ainvK, (b / torch.matmul(K.t(), u)))
    v = b / torch.matmul(K.t(), u)

    T = u * (v.t() * K)

    if not backpropT:
        T = T.detach()

    # Compute transport cost from plan and cost matrices
    E = T * Mt
    D = 2 * torch.sum(E)
    return D
```

Step 1: CFRNet Architecture

Since CFR shares the exact same network layers as TARNet, we can easily build it by inheriting from our `TARNet` class.

The only small adjustment is that our `forward` function will also return the shared representation $\Phi(X)$ so we can calculate the IPM penalty on it during training.

In [9]:
```
class CFRNet(TARNet):
    r"""
    Counterfactual Regression Network (CFRNet).
```

```
    CFRNet extends TARNet by exposing the latent representation \Phi(X) from the
    forward pass. This allows us to calculate an Integral Probability Metric (IP
    penalty (like MMD or Wasserstein) on the representation during training,
    directly regularizing the model to reduce selection bias between treated and
    """
    def forward(self, x, t):
        phi = self.phi(x)
        y0_hat = self.h0(phi)
        y1_hat = self.h1(phi)

        # Select factual prediction
        y_hat = (1 - t) * y0_hat + t * y1_hat

        # Expose phi alongside outcome predictions for balance penalty calculati
        return y_hat, y0_hat, y1_hat, phi
```

Step 2: Training CFR-MMD

Let's train our first CFR model: **CFR-MMD**! We will use the exact same dataset replication (`rep=0`) and hyperparameters, adding the linear MMD penalty directly to our factual loss.

In [36]:
```
print(f"Training CFR-MMD on Realization {rep}...")

cfr_mmd = CFRNet().to(device)
optimizer_mmd = torch.optim.Adam(cfr_mmd.parameters(), lr=1e-3, weight_decay=1e-

# CFR balance penalty hyperparameter selection
alpha = 1
p_treated = float(t_tr.mean())
iterations = 200

# Core training loop over designated iterations with MMD penalty
cfr_mmd.train()
current_iter = 0
while current_iter < iterations:
    for batch_x, batch_t, batch_y in dataloader_train:
        if current_iter >= iterations:
            break

        optimizer_mmd.zero_grad()
        y_hat, _, _, phi = cfr_mmd(batch_x, batch_t)

        # Factual MSE Loss combined with Linear MMD balance penalty
        loss_factual = tarnet_loss(batch_y, y_hat)
        loss_ipm = mmd2_lin(phi, batch_t, p_treated)
        loss = loss_factual + alpha * loss_ipm

        loss.backward()
        optimizer_mmd.step()

        if current_iter % 10 == 0:
            print("Train loss:", loss.item())
        current_iter += 1

print("CFR-MMD Training complete!")
```

```
Training CFR-MMD on Realization 0...
Train loss: 16.487049102783203
Train loss: 8.806137084960938
Train loss: 3.0248630046844482
Train loss: 2.0493979454040527
Train loss: 1.9089323282241821
Train loss: 1.5187684297561646
Train loss: 1.1665782928466797
Train loss: 1.8168340921401978
Train loss: 1.4488496780395508
Train loss: 1.5240049362182617
Train loss: 1.3659979104995728
Train loss: 0.9106789231300354
Train loss: 1.266863226890564
Train loss: 0.9706065654754639
Train loss: 0.9845130443572998
Train loss: 1.0191638469696045
Train loss: 0.96187424659729
Train loss: 1.0274070501327515
Train loss: 0.752837598323822
Train loss: 1.011200189590454
CFR-MMD Training complete!
```

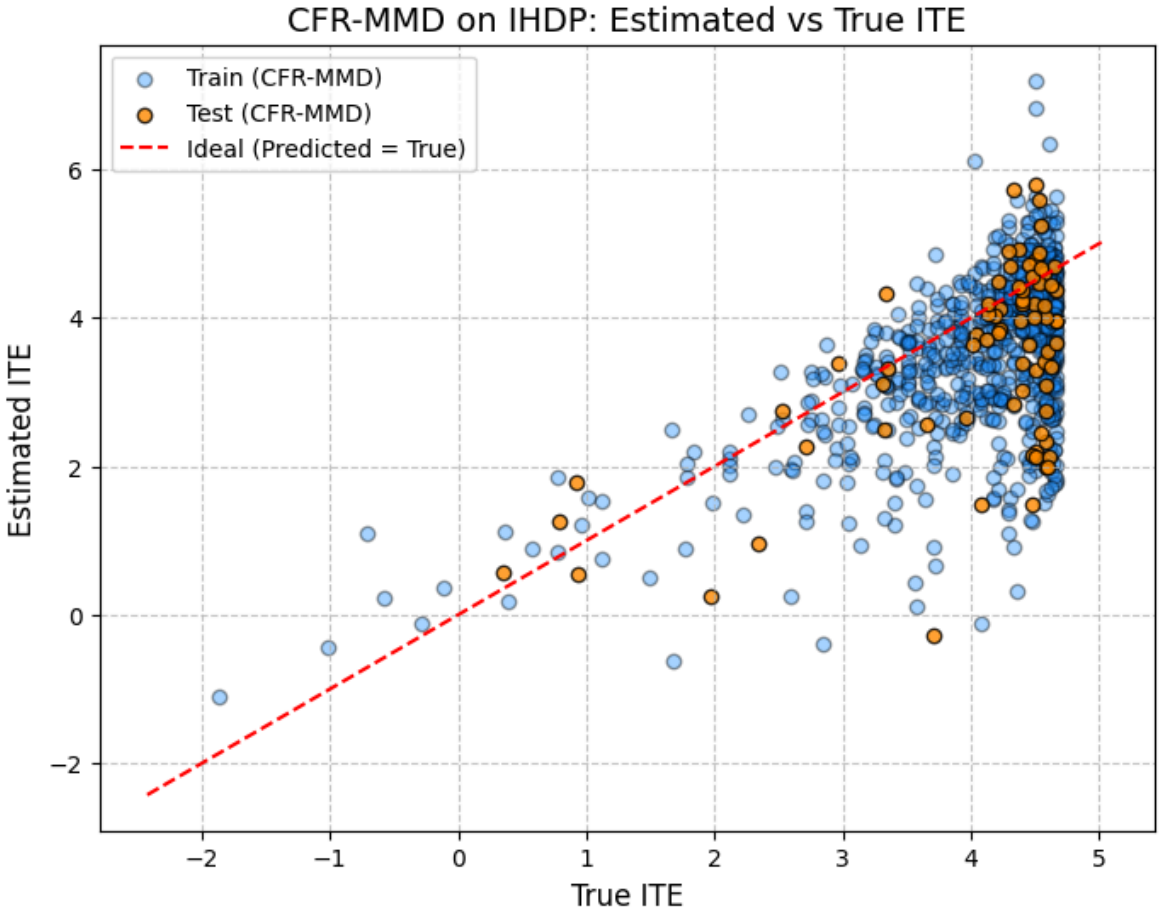
Step 3: Evaluating CFR-MMD

With our CFR-MMD model trained, let's calculate its in-sample and out-of-sample PEHE and plot its predicted ITE against the ground truth to see how the MMD penalty affects performance.

In [37]:

```python
cfr_mmd.eval()
with torch.no_grad():
    # Evaluate In-Sample outcomes
    _, y0_hat_in, y1_hat_in, _ = cfr_mmd(x_tr, t_tr)
    ite_hat_in = (y1_hat_in - y0_hat_in).cpu().numpy()

    # Evaluate Out-of-Sample outcomes
    _, y0_hat_out, y1_hat_out, _ = cfr_mmd(x_te, t_te)
    ite_hat_out = (y1_hat_out - y0_hat_out).cpu().numpy()

# Calculate PEHE metrics
pehe_in_mmd = np.mean((ite_true_in - ite_hat_in)**2)
pehe_out_mmd = np.mean((ite_true_out - ite_hat_out)**2)

print(f"CFR-MMD In-Sample PEHE (MSE):     {pehe_in_mmd:.4f}")
print(f"CFR-MMD Out-of-Sample PEHE (MSE): {pehe_out_mmd:.4f}")

# Plot Estimated vs True ITE scatter comparing outcomes under MMD penalty
plt.figure(figsize=(8, 6))
plt.scatter(ite_true_in, ite_hat_in, alpha=0.4, color='dodgerblue', edgecolor='k
plt.scatter(ite_true_out, ite_hat_out, alpha=0.8, color='darkorange', edgecolor=
plt.plot([min_val, max_val], [min_val, max_val], 'r--', label='Ideal (Predicted
plt.title(f'CFR-MMD on IHDP: Estimated vs True ITE', fontsize=14)
plt.xlabel('True ITE', fontsize=12)
plt.ylabel('Estimated ITE', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

```
CFR-MMD In-Sample PEHE (MSE):     1.2247
CFR-MMD Out-of-Sample PEHE (MSE): 1.5387
```



Step 4: Training CFR-WASS

Next, let's train **CFR-WASS** using the Wasserstein distance as the balancing penalty. This matches the exact hyperparameter setup from the original CFRNet paper for the IHDP dataset (`wass_lambda = 1.0`).

In [12]:
```
print(f"Training CFR-WASS on Realization {rep}...")

cfr_wass = CFRNet().to(device)
optimizer_wass = torch.optim.Adam(cfr_wass.parameters(), lr=1e-3, weight_decay=1

# Set balance penalty hyperparameter alpha and Wasserstein entropic lambda
alpha = 1
wass_lambda = 1.0
iterations = 200

# Core training loop over designated iterations with Wasserstein penalty
cfr_wass.train()
current_iter = 0
while current_iter < iterations:
    for batch_x, batch_t, batch_y in dataloader_train:
        if current_iter >= iterations:
            break

        optimizer_wass.zero_grad()
        y_hat, _, _, phi = cfr_wass(batch_x, batch_t)
```

```
        # Factual MSE Loss combined with Sinkhorn-Wasserstein balance penalty
        loss_factual = tarnet_loss(batch_y, y_hat)
        loss_ipm = wasserstein(phi, batch_t, p_treated, lam=wass_lambda, its=20)
        loss = loss_factual + alpha * loss_ipm

        loss.backward()
        optimizer_wass.step()
        if current_iter % 10 == 0:
            print("Train loss:", loss.item())
        current_iter += 1

print("CFR-WASS Training complete!")
```

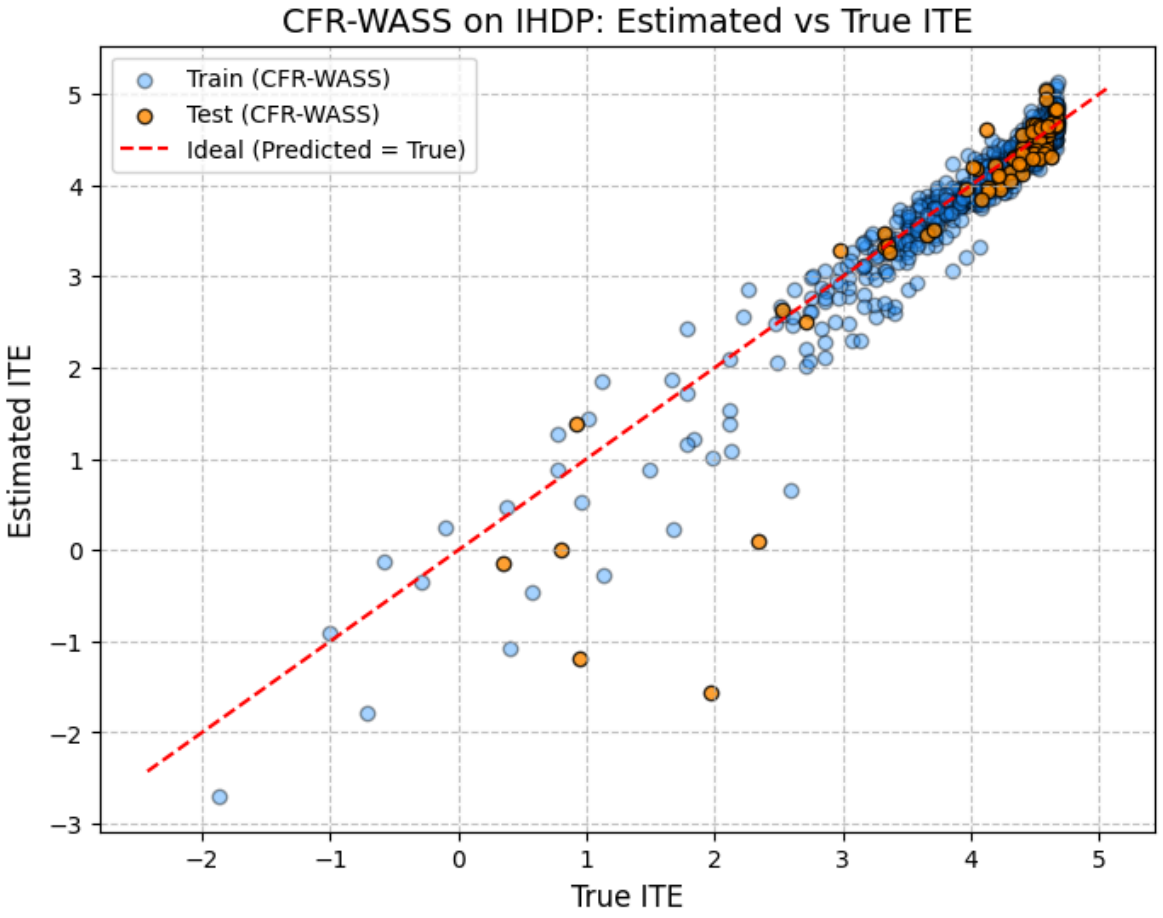
```
Training CFR-WASS on Realization 0...
Train loss: 17.0642032623291
Train loss: 10.3626070022583
Train loss: 3.4163715839385986
Train loss: 2.5774405002593994
Train loss: 3.457627296447754
Train loss: 3.0789835453033447
Train loss: 3.195030689239502
Train loss: 2.2639405727386475
Train loss: 2.214963436126709
Train loss: 2.611891746520996
Train loss: 2.4720635414123535
Train loss: 2.7190537452697754
Train loss: 2.385010242462158
Train loss: 2.4563846588134766
Train loss: 2.417505979537964
Train loss: 2.2690606117248535
Train loss: 1.8507983684539795
Train loss: 1.6387428045272827
Train loss: 2.121051788330078
Train loss: 2.397299289703369
CFR-WASS Training complete!
```

In [13]:

```
cfr_wass.eval()
with torch.no_grad():
    # Evaluate In-Sample outcomes
    _, y0_hat_in, y1_hat_in, _ = cfr_wass(x_tr, t_tr)
    ite_hat_in = (y1_hat_in - y0_hat_in).cpu().numpy()

    # Evaluate Out-of-Sample outcomes
    _, y0_hat_out, y1_hat_out, _ = cfr_wass(x_te, t_te)
    ite_hat_out = (y1_hat_out - y0_hat_out).cpu().numpy()

# Calculate PEHE metrics
pehe_in_wass = np.mean((ite_true_in - ite_hat_in)**2)
pehe_out_wass = np.mean((ite_true_out - ite_hat_out)**2)

print(f"CFR-WASS In-Sample PEHE (MSE):     {pehe_in_wass:.4f}")
print(f"CFR-WASS Out-of-Sample PEHE (MSE): {pehe_out_wass:.4f}")

# Plot Estimated vs True ITE scatter comparing outcomes under Wasserstein penalt
plt.figure(figsize=(8, 6))
plt.scatter(ite_true_in, ite_hat_in, alpha=0.4, color='dodgerblue', edgecolor='k
plt.scatter(ite_true_out, ite_hat_out, alpha=0.8, color='darkorange', edgecolor=
plt.plot([min_val, max_val], [min_val, max_val], 'r--', label='Ideal (Predicted
plt.title(f'CFR-WASS on IHDP: Estimated vs True ITE', fontsize=14)
plt.xlabel('True ITE', fontsize=12)
```

```
plt.ylabel('Estimated ITE', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

```
CFR-WASS In-Sample PEHE (MSE):     0.0650
CFR-WASS Out-of-Sample PEHE (MSE): 0.3353
```



Step 5: Effect of Alpha (α) on Representation and Estimation (Tuning & PCA for MMD & WASS)

In the previous steps, we trained CFR-MMD and CFR-WASS with a fixed balance penalty strength of $\alpha = 1.0$. However, the choice of α represents a fundamental trade-off in causal representation learning:

1. **Low α (e.g., $\alpha = 0.0$)**: The model reduces to TARNet. It focuses entirely on fitting the observed (factual) outcomes, but does not address selection bias. The treated and control distributions in the latent space $\Phi(X)$ can diverge significantly.
2. **High α**: The model heavily penalizes any discrepancy between the treated and control representation distributions, forcing them to overlap. While this reduces selection bias and improves counterfactual generalization, an excessively high α might destroy the predictive power of the representation, leading to poor outcome estimation (known as "representation collapse").

To visualize and compare these dynamics across different integral probability metrics (IPMs), we will:

1. **Tune** α across a range of values: $[0.0, 0.1, 1.0, 10.0, 100.0]$ for **both CFR-MMD and CFR-WASS** models.
2. **Compare Performance (Table)**: Compile a comparative table showing the In-Sample and Out-of-Sample PEHE (MSE) for all alpha values across both models.
3. **Plot Comparative PCA Projections**: Perform PCA on the learned representations $\Phi(X)$ for each model and each value of α to visually observe and compare how the treated and control distributions align as α increases.

In [38]:
```
import numpy as np
import pandas as pd
import torch

# Define range of alpha values to tune
alphas = [0.0, 0.1, 1.0, 10.0, 100.0]
iterations = 200

# Save data for PCA plotting (to be used in the next cell)
phi_mmd_reps = {}
phi_wass_reps = {}

results = []

# Tune CFR-MMD
print("Tuning alpha on CFR-MMD...")
for alpha in alphas:
    print(f"  Training CFR-MMD with alpha = {alpha}...")
    model = CFRNet().to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)

    model.train()
    current_iter = 0
    while current_iter < iterations:
        for batch_x, batch_t, batch_y in dataloader_train:
            if current_iter >= iterations:
                break
            optimizer.zero_grad()
            y_hat, _, _, phi = model(batch_x, batch_t)
            loss_factual = tarnet_loss(batch_y, y_hat)
            loss_ipm = mmd2_lin(phi, batch_t, p_treated)
            loss = loss_factual + alpha * loss_ipm
            loss.backward()
            optimizer.step()
            if current_iter % 10 == 0:
                print(f"    Iteration {current_iter}: Train loss = {loss.item():
            current_iter += 1

    model.eval()
    with torch.no_grad():
        _, y0_hat_in, y1_hat_in, phi_in = model(x_tr, t_tr)
        ite_hat_in = (y1_hat_in - y0_hat_in).cpu().numpy()
        ite_true_in = mu1_train_all[:, rep:rep+1] - mu0_train_all[:, rep:rep+1]
        pehe_in = np.mean((ite_true_in - ite_hat_in)**2)

        phi_mmd_reps[alpha] = phi_in.cpu().numpy()

        _, y0_hat_out, y1_hat_out, _ = model(x_te, t_te)
        ite_hat_out = (y1_hat_out - y0_hat_out).cpu().numpy()
```

```
        ite_true_out = mu1_test_all[:, rep:rep+1] - mu0_test_all[:, rep:rep+1]
        pehe_out = np.mean((ite_true_out - ite_hat_out)**2)

    results.append({
        "Model": "CFR-MMD",
        "Alpha": alpha,
        "In-Sample PEHE": round(pehe_in, 4),
        "Out-of-Sample PEHE": round(pehe_out, 4)
    })

# Tune CFR-WASS
print("\nTuning alpha on CFR-WASS...")
for alpha in alphas:
    print(f"  Training CFR-WASS with alpha = {alpha}...")
    model = CFRNet().to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
    wass_lambda = 1.0

    model.train()
    current_iter = 0
    while current_iter < iterations:
        for batch_x, batch_t, batch_y in dataloader_train:
            if current_iter >= iterations:
                break
            optimizer.zero_grad()
            y_hat, _, _, phi = model(batch_x, batch_t)
            loss_factual = tarnet_loss(batch_y, y_hat)
            loss_ipm = wasserstein(phi, batch_t, p_treated, lam=wass_lambda, its
            loss = loss_factual + alpha * loss_ipm
            loss.backward()
            optimizer.step()
            if current_iter % 10 == 0:
                print(f"    Iteration {current_iter}: Train loss = {loss.item():
            current_iter += 1

    model.eval()
    with torch.no_grad():
        _, y0_hat_in, y1_hat_in, phi_in = model(x_tr, t_tr)
        ite_hat_in = (y1_hat_in - y0_hat_in).cpu().numpy()
        ite_true_in = mu1_train_all[:, rep:rep+1] - mu0_train_all[:, rep:rep+1]
        pehe_in = np.mean((ite_true_in - ite_hat_in)**2)

        phi_wass_reps[alpha] = phi_in.cpu().numpy()

        _, y0_hat_out, y1_hat_out, _ = model(x_te, t_te)
        ite_hat_out = (y1_hat_out - y0_hat_out).cpu().numpy()
        ite_true_out = mu1_test_all[:, rep:rep+1] - mu0_test_all[:, rep:rep+1]
        pehe_out = np.mean((ite_true_out - ite_hat_out)**2)

    results.append({
        "Model": "CFR-WASS",
        "Alpha": alpha,
        "In-Sample PEHE": round(pehe_in, 4),
        "Out-of-Sample PEHE": round(pehe_out, 4)
    })

print("\nTuning complete!")

# Create and display comparison table
```

```
df_results = pd.DataFrame(results)
df_results
```

```
Tuning alpha on CFR-MMD...
  Training CFR-MMD with alpha = 0.0...
    Iteration 0: Train loss = 14.3941
    Iteration 10: Train loss = 9.2509
    Iteration 20: Train loss = 3.1622
    Iteration 30: Train loss = 1.6797
    Iteration 40: Train loss = 1.1293
    Iteration 50: Train loss = 1.5801
    Iteration 60: Train loss = 1.0029
    Iteration 70: Train loss = 0.7023
    Iteration 80: Train loss = 1.2820
    Iteration 90: Train loss = 1.1361
    Iteration 100: Train loss = 0.9861
    Iteration 110: Train loss = 1.2665
    Iteration 120: Train loss = 0.9821
    Iteration 130: Train loss = 0.7570
    Iteration 140: Train loss = 0.8562
    Iteration 150: Train loss = 1.1164
    Iteration 160: Train loss = 0.8089
    Iteration 170: Train loss = 1.0692
    Iteration 180: Train loss = 1.0953
    Iteration 190: Train loss = 0.9540
  Training CFR-MMD with alpha = 0.1...
    Iteration 0: Train loss = 13.1722
    Iteration 10: Train loss = 7.8832
    Iteration 20: Train loss = 2.8849
    Iteration 30: Train loss = 1.7003
    Iteration 40: Train loss = 1.2553
    Iteration 50: Train loss = 1.2245
    Iteration 60: Train loss = 1.0726
    Iteration 70: Train loss = 1.1938
    Iteration 80: Train loss = 1.1010
    Iteration 90: Train loss = 1.1408
    Iteration 100: Train loss = 1.0487
    Iteration 110: Train loss = 1.0916
    Iteration 120: Train loss = 1.0845
    Iteration 130: Train loss = 1.0720
    Iteration 140: Train loss = 1.2805
    Iteration 150: Train loss = 0.8768
    Iteration 160: Train loss = 1.0444
    Iteration 170: Train loss = 1.1382
    Iteration 180: Train loss = 1.0285
    Iteration 190: Train loss = 1.0442
  Training CFR-MMD with alpha = 1.0...
    Iteration 0: Train loss = 16.9640
    Iteration 10: Train loss = 8.6679
    Iteration 20: Train loss = 4.3973
    Iteration 30: Train loss = 2.1365
    Iteration 40: Train loss = 1.3539
    Iteration 50: Train loss = 1.2163
    Iteration 60: Train loss = 1.5914
    Iteration 70: Train loss = 1.3816
    Iteration 80: Train loss = 1.2611
    Iteration 90: Train loss = 1.6026
    Iteration 100: Train loss = 1.4189
    Iteration 110: Train loss = 1.5973
    Iteration 120: Train loss = 0.9771
    Iteration 130: Train loss = 0.8938
    Iteration 140: Train loss = 0.9628
    Iteration 150: Train loss = 1.0425
```

```
    Iteration 160: Train loss = 0.8269
    Iteration 170: Train loss = 1.0544
    Iteration 180: Train loss = 0.8034
    Iteration 190: Train loss = 0.9808
  Training CFR-MMD with alpha = 10.0...
    Iteration 0: Train loss = 22.3985
    Iteration 10: Train loss = 12.8559
    Iteration 20: Train loss = 3.4759
    Iteration 30: Train loss = 2.1572
    Iteration 40: Train loss = 2.0300
    Iteration 50: Train loss = 1.6123
    Iteration 60: Train loss = 1.4302
    Iteration 70: Train loss = 2.1559
    Iteration 80: Train loss = 1.4606
    Iteration 90: Train loss = 2.0770
    Iteration 100: Train loss = 1.5284
    Iteration 110: Train loss = 1.3687
    Iteration 120: Train loss = 1.1315
    Iteration 130: Train loss = 1.3464
    Iteration 140: Train loss = 0.9443
    Iteration 150: Train loss = 1.4546
    Iteration 160: Train loss = 1.4860
    Iteration 170: Train loss = 1.4004
    Iteration 180: Train loss = 1.4958
    Iteration 190: Train loss = 1.5530
  Training CFR-MMD with alpha = 100.0...
    Iteration 0: Train loss = 86.9396
    Iteration 10: Train loss = 15.2018
    Iteration 20: Train loss = 7.3551
    Iteration 30: Train loss = 4.9218
    Iteration 40: Train loss = 3.5812
    Iteration 50: Train loss = 8.0108
    Iteration 60: Train loss = 4.1260
    Iteration 70: Train loss = 2.8449
    Iteration 80: Train loss = 3.1694
    Iteration 90: Train loss = 2.9965
    Iteration 100: Train loss = 2.8939
    Iteration 110: Train loss = 3.7750
    Iteration 120: Train loss = 5.1377
    Iteration 130: Train loss = 3.0961
    Iteration 140: Train loss = 3.6344
    Iteration 150: Train loss = 4.4049
    Iteration 160: Train loss = 10.7247
    Iteration 170: Train loss = 1.6427
    Iteration 180: Train loss = 2.8798
    Iteration 190: Train loss = 6.6066

Tuning alpha on CFR-WASS...
  Training CFR-WASS with alpha = 0.0...
    Iteration 0: Train loss = 13.8844
    Iteration 10: Train loss = 8.9806
    Iteration 20: Train loss = 3.0965
    Iteration 30: Train loss = 1.5320
    Iteration 40: Train loss = 1.4278
    Iteration 50: Train loss = 1.1372
    Iteration 60: Train loss = 1.0905
    Iteration 70: Train loss = 1.2445
    Iteration 80: Train loss = 0.9766
    Iteration 90: Train loss = 0.8556
    Iteration 100: Train loss = 1.0066
```

```
  Iteration 110: Train loss = 0.8629
  Iteration 120: Train loss = 0.7956
  Iteration 130: Train loss = 0.9152
  Iteration 140: Train loss = 1.0087
  Iteration 150: Train loss = 0.8299
  Iteration 160: Train loss = 0.7330
  Iteration 170: Train loss = 1.0982
  Iteration 180: Train loss = 1.1273
  Iteration 190: Train loss = 0.8335
Training CFR-WASS with alpha = 0.1...
  Iteration 0: Train loss = 14.2922
  Iteration 10: Train loss = 7.0516
  Iteration 20: Train loss = 2.2330
  Iteration 30: Train loss = 1.6369
  Iteration 40: Train loss = 2.1791
  Iteration 50: Train loss = 1.2794
  Iteration 60: Train loss = 1.2514
  Iteration 70: Train loss = 1.2519
  Iteration 80: Train loss = 0.9066
  Iteration 90: Train loss = 1.1036
  Iteration 100: Train loss = 0.9685
  Iteration 110: Train loss = 0.8932
  Iteration 120: Train loss = 1.1740
  Iteration 130: Train loss = 1.1392
  Iteration 140: Train loss = 0.9199
  Iteration 150: Train loss = 1.0244
  Iteration 160: Train loss = 1.0255
  Iteration 170: Train loss = 1.2643
  Iteration 180: Train loss = 1.1354
  Iteration 190: Train loss = 0.9321
Training CFR-WASS with alpha = 1.0...
  Iteration 0: Train loss = 18.2405
  Iteration 10: Train loss = 8.3270
  Iteration 20: Train loss = 3.3797
  Iteration 30: Train loss = 2.7097
  Iteration 40: Train loss = 3.2605
  Iteration 50: Train loss = 3.8309
  Iteration 60: Train loss = 3.5123
  Iteration 70: Train loss = 2.5666
  Iteration 80: Train loss = 2.4277
  Iteration 90: Train loss = 2.6376
  Iteration 100: Train loss = 2.5936
  Iteration 110: Train loss = 1.8201
  Iteration 120: Train loss = 1.7335
  Iteration 130: Train loss = 1.8273
  Iteration 140: Train loss = 2.0120
  Iteration 150: Train loss = 2.1680
  Iteration 160: Train loss = 1.8983
  Iteration 170: Train loss = 2.3167
  Iteration 180: Train loss = 2.5110
  Iteration 190: Train loss = 1.4440
Training CFR-WASS with alpha = 10.0...
  Iteration 0: Train loss = 36.7863
  Iteration 10: Train loss = 12.1298
  Iteration 20: Train loss = 4.0020
  Iteration 30: Train loss = 3.8914
  Iteration 40: Train loss = 2.9851
  Iteration 50: Train loss = 2.5528
  Iteration 60: Train loss = 3.3873
  Iteration 70: Train loss = 3.2868
```

```
  Iteration 80: Train loss = 2.9303
  Iteration 90: Train loss = 4.0417
  Iteration 100: Train loss = 3.2287
  Iteration 110: Train loss = 3.3167
  Iteration 120: Train loss = 2.3349
  Iteration 130: Train loss = 3.0307
  Iteration 140: Train loss = 2.9970
  Iteration 150: Train loss = 5.0864
  Iteration 160: Train loss = 4.1519
  Iteration 170: Train loss = 8.3988
  Iteration 180: Train loss = 19.9730
  Iteration 190: Train loss = 11.1617
Training CFR-WASS with alpha = 100.0...
  Iteration 0: Train loss = 253.7470
  Iteration 10: Train loss = 27.6056
  Iteration 20: Train loss = 13.4411
  Iteration 30: Train loss = 12.4290
  Iteration 40: Train loss = 11.4993
  Iteration 50: Train loss = 9.6104
  Iteration 60: Train loss = 7.5685
  Iteration 70: Train loss = 6.7176
  Iteration 80: Train loss = 8.6370
  Iteration 90: Train loss = 7.4433
  Iteration 100: Train loss = 7.1159
  Iteration 110: Train loss = 6.1450
  Iteration 120: Train loss = 7.0075
  Iteration 130: Train loss = 5.8644
  Iteration 140: Train loss = 5.0689
  Iteration 150: Train loss = 4.8282
  Iteration 160: Train loss = 5.7280
  Iteration 170: Train loss = 4.4042
  Iteration 180: Train loss = 4.5548
  Iteration 190: Train loss = 4.9412

Tuning complete!
```

Out[38]:

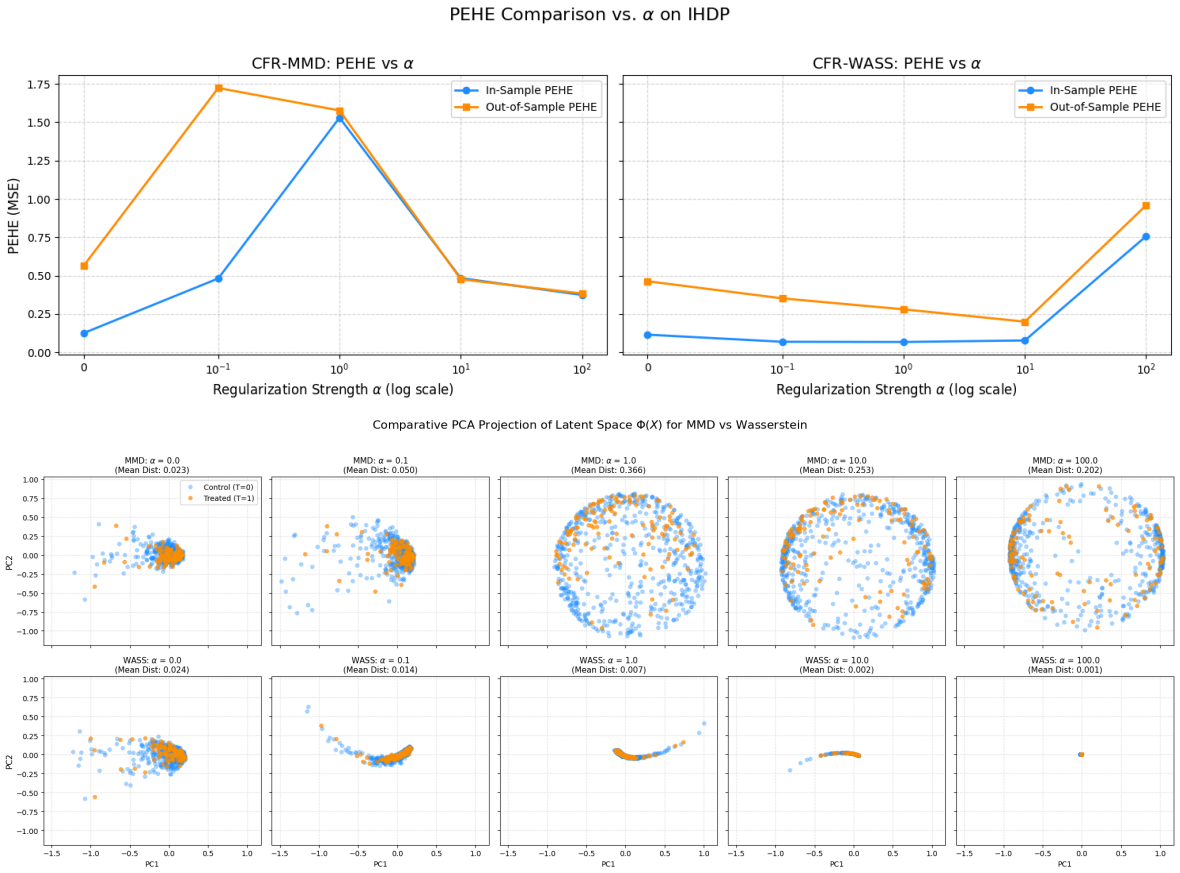
	Model	Alpha	In-Sample PEHE	Out-of-Sample PEHE
0	CFR-MMD	0.0	0.1252	0.5630
1	CFR-MMD	0.1	0.4817	1.7232
2	CFR-MMD	1.0	1.5295	1.5768
3	CFR-MMD	10.0	0.4841	0.4765
4	CFR-MMD	100.0	0.3742	0.3836
5	CFR-WASS	0.0	0.1154	0.4636
6	CFR-WASS	0.1	0.0691	0.3520
7	CFR-WASS	1.0	0.0677	0.2806
8	CFR-WASS	10.0	0.0777	0.2001
9	CFR-WASS	100.0	0.7573	0.9587

In [39]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
```

```python
# 1. Plot PEHE Comparison Curves
fig, axes = plt.subplots(1, 2, figsize=(15, 5), sharey=True)

# Extract MMD results
alphas_mmd = [r['Alpha'] for r in results if r['Model'] == 'CFR-MMD']
pehe_in_mmd = [r['In-Sample PEHE'] for r in results if r['Model'] == 'CFR-MMD']
pehe_out_mmd = [r['Out-of-Sample PEHE'] for r in results if r['Model'] == 'CFR-M

axes[0].plot(alphas_mmd, pehe_in_mmd, marker='o', label='In-Sample PEHE', color=
axes[0].plot(alphas_mmd, pehe_out_mmd, marker='s', label='Out-of-Sample PEHE', c
axes[0].set_xscale('symlog', linthresh=0.1)
axes[0].set_xlabel(r'Regularization Strength $\alpha$ (log scale)', fontsize=12)
axes[0].set_ylabel('PEHE (MSE)', fontsize=12)
axes[0].set_title(r'CFR-MMD: PEHE vs $\alpha$', fontsize=14)
axes[0].legend()
axes[0].grid(True, which="both", linestyle='--', alpha=0.5)

# Extract WASS results
alphas_wass = [r['Alpha'] for r in results if r['Model'] == 'CFR-WASS']
pehe_in_wass = [r['In-Sample PEHE'] for r in results if r['Model'] == 'CFR-WASS'
pehe_out_wass = [r['Out-of-Sample PEHE'] for r in results if r['Model'] == 'CFR-

axes[1].plot(alphas_wass, pehe_in_wass, marker='o', label='In-Sample PEHE', colo
axes[1].plot(alphas_wass, pehe_out_wass, marker='s', label='Out-of-Sample PEHE',
axes[1].set_xscale('symlog', linthresh=0.1)
axes[1].set_xlabel(r'Regularization Strength $\alpha$ (log scale)', fontsize=12)
axes[1].set_title(r'CFR-WASS: PEHE vs $\alpha$', fontsize=14)
axes[1].legend()
axes[1].grid(True, which="both", linestyle='--', alpha=0.5)

plt.suptitle(r'PEHE Comparison vs. $\alpha$ on IHDP', fontsize=16, y=1.02)
plt.tight_layout()
plt.show()

# 2. Plot Comparative PCA Grid
t_np = t_tr.cpu().numpy().flatten()
idx_t0 = (t_np == 0)
idx_t1 = (t_np == 1)

fig, axes = plt.subplots(2, len(alphas), figsize=(4.5 * len(alphas), 8.5), share

# Row 1: MMD PCA
for i, alpha in enumerate(alphas):
    ax = axes[0, i]
    phi_val = phi_mmd_reps[alpha]
    pca = PCA(n_components=2, random_state=42)
    z_pca = pca.fit_transform(phi_val)

    ax.scatter(z_pca[idx_t0, 0], z_pca[idx_t0, 1], alpha=0.4, label='Control (T=
    ax.scatter(z_pca[idx_t1, 0], z_pca[idx_t1, 1], alpha=0.7, label='Treated (T=

    dist_mean = np.linalg.norm(np.mean(phi_val[idx_t0], axis=0) - np.mean(phi_va
    ax.set_title(rf"MMD: $\alpha$ = {alpha}" + "\n" + rf"(Mean Dist: {dist_mean:
    ax.grid(True, linestyle='--', alpha=0.3)
    if i == 0:
        ax.set_ylabel('PC2', fontsize=11)
        ax.legend(loc='upper right')

# Row 2: Wasserstein PCA
for i, alpha in enumerate(alphas):
```

```
    ax = axes[1, i]
    phi_val = phi_wass_reps[alpha]
    pca = PCA(n_components=2, random_state=42)
    z_pca = pca.fit_transform(phi_val)

    ax.scatter(z_pca[idx_t0, 0], z_pca[idx_t0, 1], alpha=0.4, label='Control (T=
    ax.scatter(z_pca[idx_t1, 0], z_pca[idx_t1, 1], alpha=0.7, label='Treated (T=

    dist_mean = np.linalg.norm(np.mean(phi_val[idx_t0], axis=0) - np.mean(phi_va
    ax.set_title(rf"WASS: $\alpha$ = {alpha}" + "\n" + rf"(Mean Dist: {dist_mean
    ax.grid(True, linestyle='--', alpha=0.3)
    ax.set_xlabel('PC1', fontsize=10)
    if i == 0:
        ax.set_ylabel('PC2', fontsize=11)

plt.suptitle(r'Comparative PCA Projection of Latent Space $\Phi(X)$ for MMD vs W
plt.tight_layout()
plt.show()
```



As the visualizations demonstrate, relying on MMD can inadvertently degrade performance. To minimize the MMD penalty, the network may project the data outward into a dispersed, spherical structure. While this technically satisfies the condition of matching group means, it fractures the underlying covariate patterns necessary for accurate outcome prediction. In contrast, the Wasserstein metric seems to have accounted for the underlying geometry of the data better.

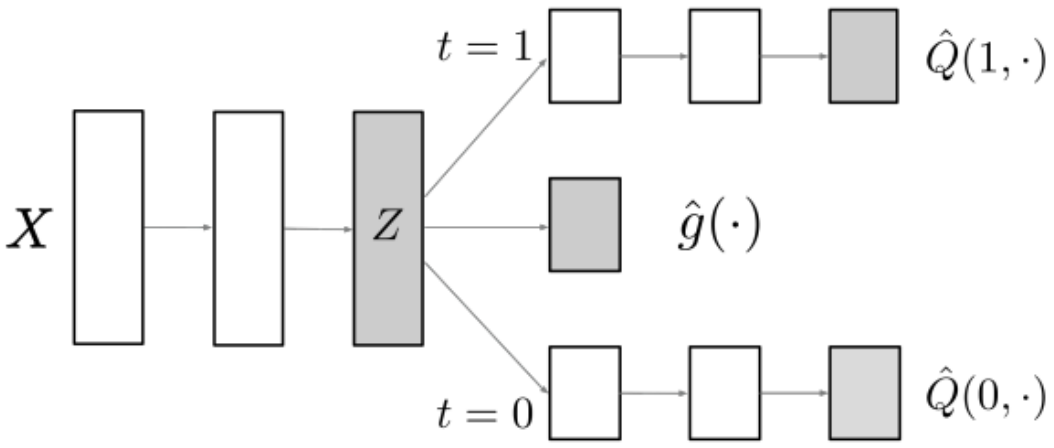
The results also highlight an interesting pattern as seen in high alpha settings for WASS. If the penalty parameter for distribution matching is set too high, the network may resort to mode collapse, mapping all treated and control units to a single point.

5. Dragonnet

Intuition & Formulation

While CFR focuses on forcing the representation of treated and control groups to overlap by penalizing their divergence, it runs the risk of discarding *confounding features* (features that are highly predictive of both the treatment decision and the outcome) if the IPM penalty is set too high.

Dragonnet takes a completely different, clever approach based on causal sufficiency. Instead of an adversarial balance penalty, Dragonnet adds a **propensity head** to the shared representation $\Phi(X)$ to explicitly predict the probability of treatment assignment $P(T = 1 \mid X)$.



The propensity prediction acts as a powerful regularizer:

$$\mathcal{L}_{Dragonnet} = \mathcal{L}_{TARNet} + \beta \cdot \text{CrossEntropy}(T_i, p(\Phi(X_i)))$$

By forcing the shared representation to predict treatment assignment, Dragonnet guarantees that all features that confound the treatment decision are actively preserved in the representation space.

Why It Excels

Dragonnet is a powerhouse for **statistical sufficiency** and highly efficient estimates of the Average Treatment Effect (ATE). It prevents the network from discarding vital confounding variables, ensuring that both the propensity score and outcome models are jointly optimized on a unified covariate space.

In [40]:
```
class Dragonnet(nn.Module):
    r"""
    Dragonnet Network Architecture.

    Dragonnet uses three heads branching from a unified shared representation \P
```

```
    - h_0: Predicts control outcome potential outcome Y(0).
    - h_1: Predicts treated outcome potential outcome Y(1).
    - propensity_head: Predicts treatment assignment probability (propensity sco

    This propensity head acts as a targeted regularizer, forcing the shared repr
    to retain features that predict treatment selection, thereby safeguarding ke
    """
    def __init__(self):
        super().__init__()
        self.phi = SharedRepresentation(input_dim=25, hidden_dim=200)
        self.h0 = HypothesisHead(input_dim=200, hidden_dim=100)
        self.h1 = HypothesisHead(input_dim=200, hidden_dim=100)

        # Propensity score head: 2-layer MLP predicting treatment assignment
        self.propensity_head = nn.Sequential(
            nn.Linear(200, 100),
            nn.ELU(),
            nn.Linear(100, 1)
        )

    def forward(self, x, t):
        phi = self.phi(x)
        y0_hat = self.h0(phi)
        y1_hat = self.h1(phi)
        t_logits = self.propensity_head(phi)

        # Factual prediction based on treatment assignment
        y_hat = (1 - t) * y0_hat + t * y1_hat
        return y_hat, y0_hat, y1_hat, t_logits, phi

def dragonnet_loss(y_true, y_hat, t_true, t_logits, beta=1.0):
    r"""
    Computes the joint objective loss function for Dragonnet.

    This loss consists of two parts:
    1. Factual regression loss (MSE) on the outcome.
    2. Binary Cross Entropy (BCE) loss on the propensity head's treatment predic
    """
    mse_criterion = nn.MSELoss()
    loss_factual = mse_criterion(y_hat, y_true)

    # Sigmoid activation and BCE combined for numerical stability
    bce_criterion = nn.BCEWithLogitsLoss()
    loss_propensity = bce_criterion(t_logits, t_true)

    # Scale propensity loss by beta regularizer weight
    total_loss = loss_factual + beta * loss_propensity
    return total_loss, loss_factual, loss_propensity
```

Step 1: Training Dragonnet

Let's initialize `Dragonnet` and train it on the same realization (`rep=0`) for exactly 3000 iterations using the Adam optimizer. We'll set $\beta = 1.0$ and see how it works.

In [29]:
```
print(f"Training Dragonnet on Realization {rep}...")

dragonnet = Dragonnet().to(device)
optimizer_dragon = torch.optim.Adam(dragonnet.parameters(), lr=1e-3, weight_deca
```

```
# Beta weight controls propensity head targeted regularization penalty
beta_val = 1.0
iterations = 100

# Core training loop over designated iterations with propensity score regulariza
dragonnet.train()
current_iter = 0
while current_iter < iterations:
    for batch_x, batch_t, batch_y in dataloader_train:
        if current_iter >= iterations:
            break

        optimizer_dragon.zero_grad()
        y_hat, _, _, t_logits, _ = dragonnet(batch_x, batch_t)

        # Joint outcome fit + propensity targeted regularization loss
        loss, _, _ = dragonnet_loss(batch_y, y_hat, batch_t, t_logits, beta=beta
        loss.backward()
        optimizer_dragon.step()

        if current_iter % 10 == 0:
            print("Train loss:", loss.item())
        current_iter += 1

print("Dragonnet Training complete!")
```

```
Training Dragonnet on Realization 0...
Train loss: 13.660888671875
Train loss: 8.813934326171875
Train loss: 3.3103156089782715
Train loss: 1.9626524448394775
Train loss: 2.1598715782165527
Train loss: 1.549499750137329
Train loss: 1.3784072399139404
Train loss: 1.7470121383666992
Train loss: 1.4171714782714844
Train loss: 1.209641456604004
Dragonnet Training complete!
```

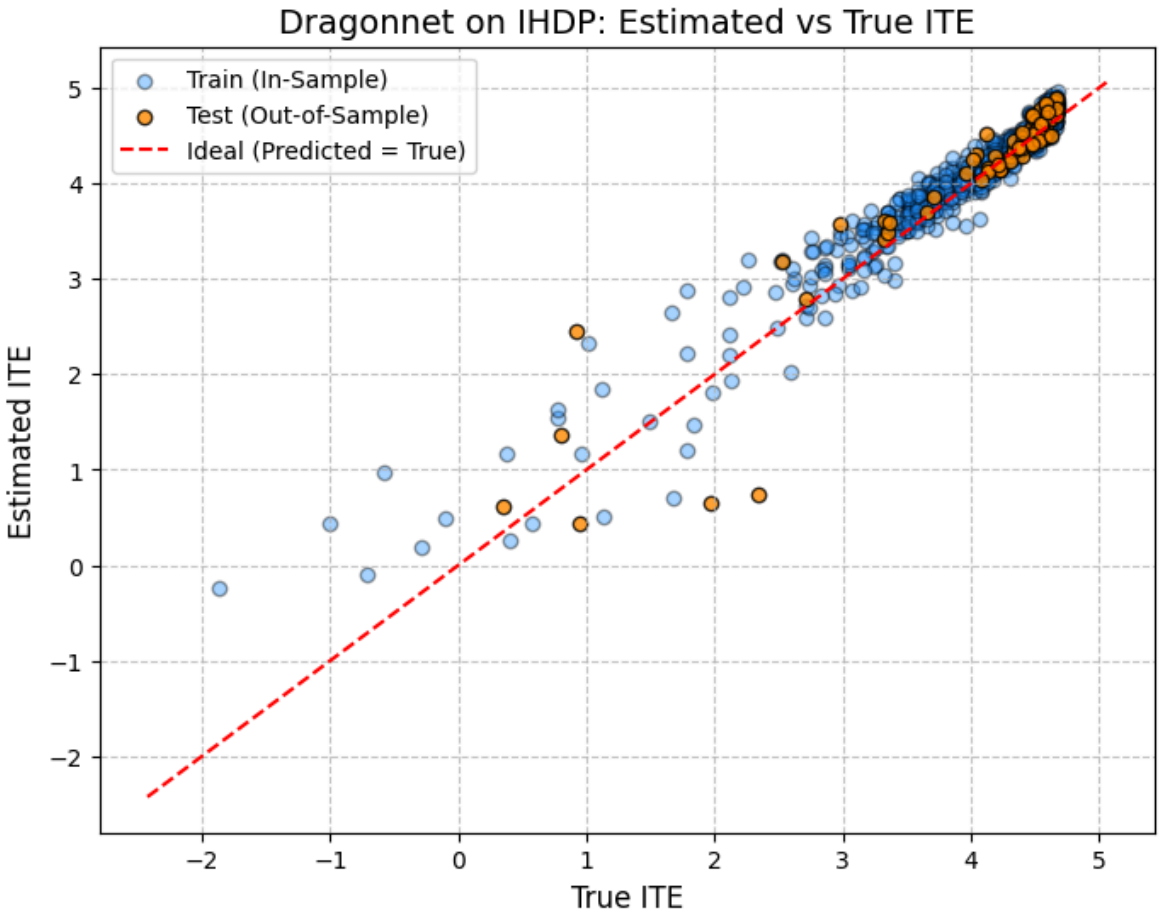
Step 2: Evaluating Dragonnet

Let's compute our in-sample and out-of-sample PEHE for Dragonnet and plot the estimated ITE against the true values to evaluate its prediction quality.

In [30]:
```
dragonnet.eval()
with torch.no_grad():
    # Evaluate In-Sample outcomes
    _, y0_hat_in, y1_hat_in, _, _ = dragonnet(x_tr, t_tr)
    ite_hat_in = (y1_hat_in - y0_hat_in).cpu().numpy()

    # Evaluate Out-of-Sample outcomes
    _, y0_hat_out, y1_hat_out, _, _ = dragonnet(x_te, t_te)
    ite_hat_out = (y1_hat_out - y0_hat_out).cpu().numpy()

# Calculate PEHE metrics
pehe_in_dragon = np.mean((ite_true_in - ite_hat_in)**2)
pehe_out_dragon = np.mean((ite_true_out - ite_hat_out)**2)
```

```
print(f"Dragonnet In-Sample PEHE (MSE):     {pehe_in_dragon:.4f}")
print(f"Dragonnet Out-of-Sample PEHE (MSE): {pehe_out_dragon:.4f}")
print(f"Dragonnet Out-of-Sample RMSE:       {np.sqrt(pehe_out_dragon):.4f}")

# Plot Estimated vs True ITE scatter comparing outcomes under targeted regulariz
plt.figure(figsize=(8, 6))
plt.scatter(ite_true_in, ite_hat_in, alpha=0.4, color='dodgerblue', edgecolor='k
plt.scatter(ite_true_out, ite_hat_out, alpha=0.8, color='darkorange', edgecolor=
plt.plot([min_val, max_val], [min_val, max_val], 'r--', label='Ideal (Predicted
plt.title(f'Dragonnet on IHDP: Estimated vs True ITE', fontsize=14)
plt.xlabel('True ITE', fontsize=12)
plt.ylabel('Estimated ITE', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

```
Dragonnet In-Sample PEHE (MSE):     0.0552
Dragonnet Out-of-Sample PEHE (MSE): 0.1241
Dragonnet Out-of-Sample RMSE:       0.3522
```



Step 3: Tuning Beta (β) for Dragonnet

In the previous steps, we trained `Dragonnet` with a fixed propensity head regularization weight of $\beta = 1.0$. The choice of β represents a key hyperparameter in Dragonnet that controls the trade-off between outcome fitting and propensity score estimation:

1. **Low β (e.g., $\beta = 0.0$)**: The model reduces to a TARNet architecture with an unregularized propensity head. In this case, the shared representation does not explicitly receive targeted propensity score regularization, which can fail to actively preserve confounding variables.

2. **Optimal** β: By setting an appropriate β, the propensity head acts as a targeted regularizer, forcing the shared representation $\Phi(X)$ to retain and emphasize features that predict treatment assignment. This ensures causal sufficiency and controls confounding without compromising outcome modeling.
3. **High** β: If β is set too high, the network focuses excessively on predicting treatment assignment at the expense of fitting the potential outcomes, potentially degrading the PEHE (CATE estimation) performance.

To investigate the effect of β on Dragonnet, we will:

1. **Tune** β across a range of values: $[0.0, 0.1, 0.5, 1.0, 5.0, 10.0]$.
2. **Compare Performance**: Compile a table showing the In-Sample and Out-of-Sample PEHE (MSE) for all β values.
3. **Plot Comparative PCA Projections**: Perform PCA on the learned representations $\Phi(X)$ for each value of β to observe how the treatment and control distributions align in the representation space under different propensity regularization strengths.

In [41]:
```
import numpy as np
import pandas as pd
import torch

# Define range of beta values to tune
betas = [0.0, 0.1, 0.5, 1.0, 5.0, 10.0]

# Save data for PCA plotting (to be used in the next cell)
phi_dragon_reps = {}

results_dragon = []
iterations = 100

print("Tuning beta on Dragonnet...")
for beta in betas:
    print(f"  Training Dragonnet with beta = {beta}...")
    model = Dragonnet().to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)

    model.train()
    current_iter = 0
    while current_iter < iterations:
        for batch_x, batch_t, batch_y in dataloader_train:
            if current_iter >= iterations:
                break
            optimizer.zero_grad()
            y_hat, _, _, t_logits, _ = model(batch_x, batch_t)

            # Joint outcome fit + propensity targeted regularization loss
            loss, _, _ = dragonnet_loss(batch_y, y_hat, batch_t, t_logits, beta=
            loss.backward()
            optimizer.step()

            if current_iter % 10 == 0:
                print(f"    Iteration {current_iter}: Train loss = {loss.item():
            current_iter += 1

    model.eval()
    with torch.no_grad():
```

```
        _, y0_hat_in, y1_hat_in, _, phi_in = model(x_tr, t_tr)
        ite_hat_in = (y1_hat_in - y0_hat_in).cpu().numpy()
        ite_true_in = mu1_train_all[:, rep:rep+1] - mu0_train_all[:, rep:rep+1]
        pehe_in = np.mean((ite_true_in - ite_hat_in)**2)

        phi_dragon_reps[beta] = phi_in.cpu().numpy()

        _, y0_hat_out, y1_hat_out, _, _ = model(x_te, t_te)
        ite_hat_out = (y1_hat_out - y0_hat_out).cpu().numpy()
        ite_true_out = mu1_test_all[:, rep:rep+1] - mu0_test_all[:, rep:rep+1]
        pehe_out = np.mean((ite_true_out - ite_hat_out)**2)

    results_dragon.append({
        "Beta": beta,
        "In-Sample PEHE": round(pehe_in, 4),
        "Out-of-Sample PEHE": round(pehe_out, 4)
    })

print("\nTuning complete!")

# Create and display comparison table
df_results_dragon = pd.DataFrame(results_dragon)
df_results_dragon
```

```
Tuning beta on Dragonnet...
  Training Dragonnet with beta = 0.0...
    Iteration 0: Train loss = 12.6068
    Iteration 10: Train loss = 7.8410
    Iteration 20: Train loss = 2.3728
    Iteration 30: Train loss = 1.6627
    Iteration 40: Train loss = 1.1829
    Iteration 50: Train loss = 0.9674
    Iteration 60: Train loss = 1.0471
    Iteration 70: Train loss = 1.0547
    Iteration 80: Train loss = 1.2565
    Iteration 90: Train loss = 1.3323
  Training Dragonnet with beta = 0.1...
    Iteration 0: Train loss = 16.0118
    Iteration 10: Train loss = 5.3855
    Iteration 20: Train loss = 2.4616
    Iteration 30: Train loss = 1.6118
    Iteration 40: Train loss = 1.8281
    Iteration 50: Train loss = 1.1851
    Iteration 60: Train loss = 1.2968
    Iteration 70: Train loss = 1.1903
    Iteration 80: Train loss = 0.9061
    Iteration 90: Train loss = 1.0437
  Training Dragonnet with beta = 0.5...
    Iteration 0: Train loss = 14.8851
    Iteration 10: Train loss = 8.1421
    Iteration 20: Train loss = 4.0114
    Iteration 30: Train loss = 1.7751
    Iteration 40: Train loss = 1.6160
    Iteration 50: Train loss = 1.1011
    Iteration 60: Train loss = 1.2344
    Iteration 70: Train loss = 1.0867
    Iteration 80: Train loss = 1.2199
    Iteration 90: Train loss = 1.1481
  Training Dragonnet with beta = 1.0...
    Iteration 0: Train loss = 11.9623
    Iteration 10: Train loss = 8.4613
    Iteration 20: Train loss = 2.8986
    Iteration 30: Train loss = 2.3111
    Iteration 40: Train loss = 2.1350
    Iteration 50: Train loss = 1.5406
    Iteration 60: Train loss = 1.3629
    Iteration 70: Train loss = 1.5557
    Iteration 80: Train loss = 1.5199
    Iteration 90: Train loss = 1.7349
  Training Dragonnet with beta = 5.0...
    Iteration 0: Train loss = 16.8607
    Iteration 10: Train loss = 12.4501
    Iteration 20: Train loss = 4.7383
    Iteration 30: Train loss = 4.5092
    Iteration 40: Train loss = 3.6318
    Iteration 50: Train loss = 3.0893
    Iteration 60: Train loss = 3.1416
    Iteration 70: Train loss = 3.4505
    Iteration 80: Train loss = 3.2357
    Iteration 90: Train loss = 4.2868
  Training Dragonnet with beta = 10.0...
    Iteration 0: Train loss = 22.5401
    Iteration 10: Train loss = 12.9435
    Iteration 20: Train loss = 7.9645
```

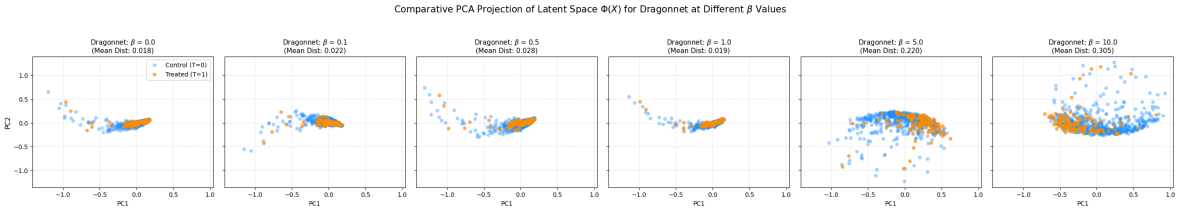
```
Iteration 30: Train loss = 6.5373
Iteration 40: Train loss = 5.4597
Iteration 50: Train loss = 5.7274
Iteration 60: Train loss = 5.5031
Iteration 70: Train loss = 5.7436
Iteration 80: Train loss = 5.6427
Iteration 90: Train loss = 4.5499

Tuning complete!
```

Out[41]:

	Beta	In-Sample PEHE	Out-of-Sample PEHE
0	0.0	0.0500	0.2119
1	0.1	0.0523	0.2244
2	0.5	0.1342	0.5680
3	1.0	0.0682	0.1978
4	5.0	0.1804	0.3159
5	10.0	0.2051	0.3423

In [42]:

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Extract treatment indicator for training samples
t_np = t_tr.cpu().numpy().flatten()
idx_t0 = (t_np == 0)
idx_t1 = (t_np == 1)

# Plot PCA for each beta value in a single row
fig, axes = plt.subplots(1, len(betas), figsize=(4.5 * len(betas), 4.5), sharex=

for i, beta in enumerate(betas):
    ax = axes[i]
    phi_val = phi_dragon_reps[beta]
    pca = PCA(n_components=2, random_state=42)
    z_pca = pca.fit_transform(phi_val)

    ax.scatter(z_pca[idx_t0, 0], z_pca[idx_t0, 1], alpha=0.4, label='Control (T=
    ax.scatter(z_pca[idx_t1, 0], z_pca[idx_t1, 1], alpha=0.7, label='Treated (T=

    dist_mean = np.linalg.norm(np.mean(phi_val[idx_t0], axis=0) - np.mean(phi_va
    ax.set_title(rf"Dragonnet: $\beta$ = {beta}" + "\n" + rf"(Mean Dist: {dist_m
    ax.grid(True, linestyle='--', alpha=0.3)
    ax.set_xlabel('PC1', fontsize=10)
    if i == 0:
        ax.set_ylabel('PC2', fontsize=11)
        ax.legend(loc='upper right')

plt.suptitle(r'Comparative PCA Projection of Latent Space $\Phi(X)$ for Dragonne
plt.tight_layout()
plt.show()
```



6. Visualizing the Learned Representations

To wrap things up and truly understand how representation learning differs across all four architectures, let's visualize the high-dimensional latent representation $\Phi(X)$ that each model learns!

First, let's look at a comprehensive comparison of all the models. We will evaluate `TARNet`, `CFR-MMD` (using its optimal α), `CFR-WASS` (using its optimal α), and `Dragonnet` (using its optimal β). The table below compares the in-sample and out-of-sample PEHE for all four models.

In [43]:
```
import numpy as np
import pandas as pd
import torch

# Evaluate TARNet PEHE
tarnet.eval()
with torch.no_grad():
    _, y0_hat_in, y1_hat_in = tarnet(x_tr, t_tr)
    ite_hat_in = (y1_hat_in - y0_hat_in).cpu().numpy()
    ite_true_in = mu1_train_all[:, rep:rep+1] - mu0_train_all[:, rep:rep+1]
    tarnet_pehe_in = np.mean((ite_true_in - ite_hat_in)**2)

    _, y0_hat_out, y1_hat_out = tarnet(x_te, t_te)
    ite_hat_out = (y1_hat_out - y0_hat_out).cpu().numpy()
    ite_true_out = mu1_test_all[:, rep:rep+1] - mu0_test_all[:, rep:rep+1]
    tarnet_pehe_out = np.mean((ite_true_out - ite_hat_out)**2)

# Find best alpha for CFR-MMD from results
best_mmd_row = df_results[df_results["Model"] == "CFR-MMD"].loc[
    df_results[df_results["Model"] == "CFR-MMD"]["Out-of-Sample PEHE"].idxmin()
]
best_alpha_mmd = best_mmd_row["Alpha"]
pehe_in_mmd = best_mmd_row["In-Sample PEHE"]
pehe_out_mmd = best_mmd_row["Out-of-Sample PEHE"]

# Find best alpha for CFR-WASS from results
best_wass_row = df_results[df_results["Model"] == "CFR-WASS"].loc[
    df_results[df_results["Model"] == "CFR-WASS"]["Out-of-Sample PEHE"].idxmin()
]
best_alpha_wass = best_wass_row["Alpha"]
pehe_in_wass = best_wass_row["In-Sample PEHE"]
pehe_out_wass = best_wass_row["Out-of-Sample PEHE"]

# Find best beta for Dragonnet from results
best_dragon_row = df_results_dragon.loc[df_results_dragon["Out-of-Sample PEHE"].
best_beta_dragon = best_dragon_row["Beta"]
pehe_in_dragon = best_dragon_row["In-Sample PEHE"]
pehe_out_dragon = best_dragon_row["Out-of-Sample PEHE"]
```

```
# Compile comparative data
comparison_data = [
    {
        "Model": "TARNet",
        "Best Hyperparameter": "N/A",
        "In-Sample PEHE": round(tarnet_pehe_in, 4),
        "Out-of-Sample PEHE": round(tarnet_pehe_out, 4)
    },
    {
        "Model": "CFR-MMD",
        "Best Hyperparameter": f"alpha = {best_alpha_mmd}",
        "In-Sample PEHE": pehe_in_mmd,
        "Out-of-Sample PEHE": pehe_out_mmd
    },
    {
        "Model": "CFR-WASS",
        "Best Hyperparameter": f"alpha = {best_alpha_wass}",
        "In-Sample PEHE": pehe_in_wass,
        "Out-of-Sample PEHE": pehe_out_wass
    },
    {
        "Model": "Dragonnet",
        "Best Hyperparameter": f"beta = {best_beta_dragon}",
        "In-Sample PEHE": pehe_in_dragon,
        "Out-of-Sample PEHE": pehe_out_dragon
    }
]

df_comparison = pd.DataFrame(comparison_data)
df_comparison
```

Out[43]:

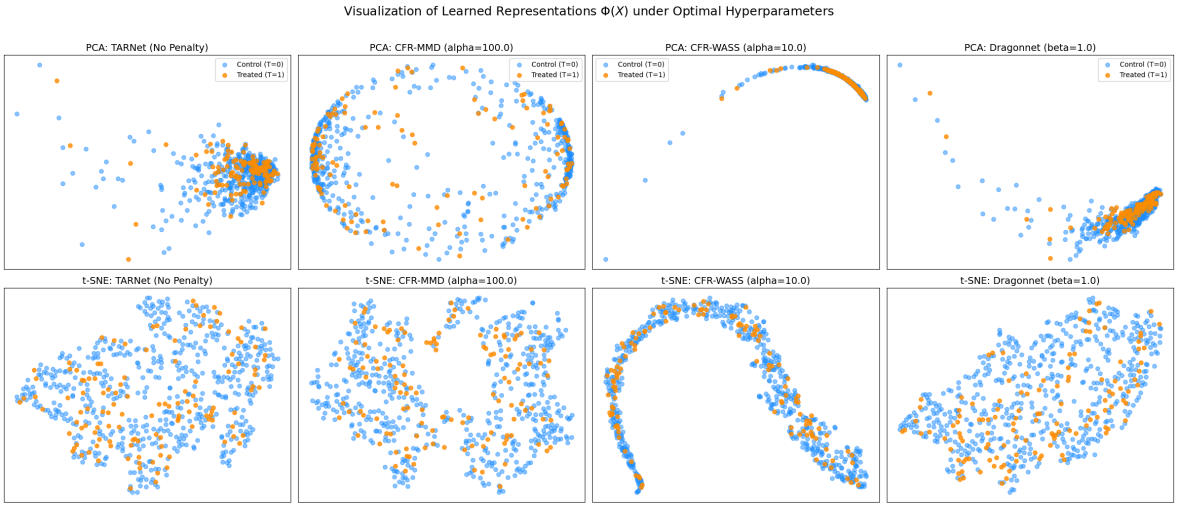
	Model	Best Hyperparameter	In-Sample PEHE	Out-of-Sample PEHE
0	TARNet	N/A	0.0992	0.4994
1	CFR-MMD	alpha = 100.0	0.3742	0.3836
2	CFR-WASS	alpha = 10.0	0.0777	0.2001
3	Dragonnet	beta = 1.0	0.0682	0.1978

Now, let's extract the learned latent representations $\Phi(X)$ of each model at their optimal hyperparameters. We will apply **PCA** (Principal Component Analysis) and **t-SNE** to reduce them to 2 dimensions for visual inspection and comparison.

In [44]:
```
from sklearn.manifold import TSNE
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Extract latent representation phi from each trained model architecture at thei
with torch.no_grad():
    phi_tarnet = tarnet.phi(x_tr).cpu().numpy()

    # Use the best-performing hyperparameter representations cached during tunin
    phi_mmd = phi_mmd_reps[best_alpha_mmd]
    phi_wass = phi_wass_reps[best_alpha_wass]
    phi_dragon = phi_dragon_reps[best_beta_dragon]
```

```python
# Apply Dimensionality Reduction (PCA and t-SNE) to map features down to 2D
print("Running PCA & t-SNE... (t-SNE takes a few seconds)")

pca = PCA(n_components=2, random_state=42)
z_pca_tarnet = pca.fit_transform(phi_tarnet)
z_pca_mmd = pca.fit_transform(phi_mmd)
z_pca_wass = pca.fit_transform(phi_wass)
z_pca_dragon = pca.fit_transform(phi_dragon)

tsne = TSNE(n_components=2, random_state=42, perplexity=30)
z_tsne_tarnet = tsne.fit_transform(phi_tarnet)
z_tsne_mmd = tsne.fit_transform(phi_mmd)
z_tsne_wass = tsne.fit_transform(phi_wass)
z_tsne_dragon = tsne.fit_transform(phi_dragon)

# Split embeddings indices by observed factual treatment to plot distinct distri
t_np = t_tr.cpu().numpy().flatten()
idx_t0 = (t_np == 0)
idx_t1 = (t_np == 1)

fig, axes = plt.subplots(2, 4, figsize=(24, 10))
titles = [
    'TARNet (No Penalty)',
    f'CFR-MMD (alpha={best_alpha_mmd})',
    f'CFR-WASS (alpha={best_alpha_wass})',
    f'Dragonnet (beta={best_beta_dragon})'
]
pca_embeddings = [z_pca_tarnet, z_pca_mmd, z_pca_wass, z_pca_dragon]
tsne_embeddings = [z_tsne_tarnet, z_tsne_mmd, z_tsne_wass, z_tsne_dragon]

# Plot linear PCA projections for representation comparison
for ax, z, title in zip(axes[0], pca_embeddings, titles):
    ax.scatter(z[idx_t0, 0], z[idx_t0, 1], alpha=0.5, label='Control (T=0)', col
    ax.scatter(z[idx_t1, 0], z[idx_t1, 1], alpha=0.8, label='Treated (T=1)', col
    ax.set_title(f"PCA: {title}", fontsize=14)
    ax.set_xticks([])
    ax.set_yticks([])
    ax.legend()

# Plot non-linear t-SNE neighborhood mappings
for ax, z, title in zip(axes[1], tsne_embeddings, titles):
    ax.scatter(z[idx_t0, 0], z[idx_t0, 1], alpha=0.5, label='Control (T=0)', col
    ax.scatter(z[idx_t1, 0], z[idx_t1, 1], alpha=0.8, label='Treated (T=1)', col
    ax.set_title(f"t-SNE: {title}", fontsize=14)
    ax.set_xticks([])
    ax.set_yticks([])

plt.suptitle('Visualization of Learned Representations $\\Phi(X)$ under Optimal
plt.tight_layout()
plt.show()
```

```
Running PCA & t-SNE... (t-SNE takes a few seconds)
```



The final visualization illustrate how different regularization strategies structurally alter the latent feature space $\Phi(X)$ to achieve covariate balance. Comparing the learned representations, we see:

The baseline TARNet, which operates without a distributional penalty, maps the data into overlapping but structurally distinct clusters for the treated and control groups.

Applying a heavy MMD penalty ($\alpha = 100.0$) forces the network to artificially disperse the representations into a hollow, fragmented ring. This severe distortion achieves mathematical balance but fractures the underlying predictive manifold.

In contrast, the Wasserstein metric (CFR-WASS, $\alpha = 10.0$) integrates the two groups by smoothly mapping them onto a dense, continuous curve, effectively aligning the distributions while preserving the essential geometric structure needed for reliable outcome prediction.

Dragonnet ($\beta = 1.0$) demonstrates a distinct approach via targeted regularization. Rather than explicitly minimizing a global distance metric between the two domains, it shapes the representation to ensure sufficiency for propensity score estimation alongside outcome prediction. The resulting manifold seem to exhibit strong cohesion between the treated and control units without the extreme geometric constraints seen in the CFR models, yielding a balanced and structurally sound foundation for the subsequent causal estimators.

Conclusion

In this walkthrough, we see that Dragonnet achieved the best overall predictive performance. The counterfactual regression models, CFR-WASS and CFR-MMD, followed behind, offering slight improvements over the baseline TARNet architecture.

The visualizations of the learned representations offer helpful intuition for why these performance differences exist. The 2D PCA plots demonstrate that the Wasserstein penalty (CFR-WASS) aggressively nudges the data into a distinct curved line, heavily constraining the latent space to force the treated and control units into similar distributions. Dragonnet appears to achieve a similar degree of functional overlap

between the two groups, but it manages to preserve noticeably more variance from the original feature space. This capacity to achieve balance while retaining richer underlying data variance is likely a primary reason why Dragonnet yields superior treatment effect estimates in this setting.

One critical factor observed during these experiments was the sensitivity of these models to the number of training steps. Initially, when the models were trained for 3000 iterations, the out-of-sample PEHE degraded severely, exploding to over 2. By lowering the training duration to 200 iterations, the PEHE dropped significantly to less than 1 across all architectures. This suggests a strong tendency for these neural networks to overfit the factual outcomes at the expense of counterfactual generalization. A deeper investigation into training dynamics, early stopping, and hyperparameter stability is not fully explored in this tutorial and is left as an exercise for the reader.

Further Reading

To explore the theoretical foundations and mathematical proofs behind the architectures discussed in this notebook, the following foundational papers provide excellent starting points:

- Hill, J. L. (2011). Bayesian nonparametric modeling for causal inference. Journal of Computational and Graphical Statistics, 20(1), 217-240. https://doi.org/10.1198/jcgs.2010.08162
- Shalit, U., Johansson, F. D., & Sontag, D. (2017, July). Estimating individual treatment effect: generalization bounds and algorithms. In International conference on machine learning (pp. 3076-3085). PMLR. https://doi.org/10.48550/arxiv.1606.03976
- Shi, C., Blei, D., & Veitch, V. (2019). Adapting neural networks for the estimation of treatment effects. Advances in neural information processing systems, 32. https://doi.org/10.48550/arxiv.1906.02120